

Towards Efficient BERT Inference on x86 CPUs with AITemplate

摘要 | 本專題在原先以 CUDA/HIP 為主要執行目標的 AITemplate 中加入 x86 CPU backend，並以 BERT-family inference 驗證其可行性。實作包含 CPU code generation、XNNPACK 整合，以及 static weight prepacking、temporary buffer reduction 與 operator fusion。實驗結果顯示，BERT-base 與 BERT-large 的推論延遲皆低於 PyTorch CPU baseline，其中 BERT-large 的 peak RSS 約降低 19.5%

1. 研究動機與問題

AITemplate 是 Meta 開源的 deep learning compiler，原始公開架構主要以 NVIDIA CUDA 與 AMD ROCm GPU 為執行目標。其 frontend 負責 graph transformation，backend 則依照目標硬體產生對應的 C++ 與 kernel code。[1] 由於原始 operator implementation、backend 與部分 runtime interface 都建立在 GPU execution 的假設上，因此若要支援 x86 CPU，除了新增 CPU kernel，也必須處理 code generation、runtime 介面與記憶體使用方式。

深度學習編譯器通常還需要處理不同硬體上的效能差異。比如 TVM 透過 graph-level 與 operator-level optimization 支援多種硬體 target；oneDNN Graph Compiler 則將 optimized kernels 與 graph compilation 結合，並針對 constant weights、operator fusion 與 memory-buffer reuse 進行最佳化。[2][3] 這些工作顯示，CPU inference 的效能不只取決於單一 kernel，也會受到整體 graph 與資料搬移成本影響。

本專題因此以保留 AITemplate 原有 graph compilation 與 code-generation 架構為前提，加入 x86 CPU backend，並聚焦以下三個問題：

- 如何將 BERT 所需 operators 映射到 x86 CPU，並沿用 AITemplate 既有的編譯與執行流程。
- 在使用 optimized CPU kernels 後，static weight preparation、temporary memory 與額外 tensor pass 是否仍會影響 end-to-end latency。
- 能否利用編譯時已知的 tensor shape、constant weight 與 graph dependency，進一步減少上述 overhead。

BERT 被選為主要 workload，除了原始專案已提供測試程式碼範本，也因為其 encoder 同時包含大量 Fully Connected / GEMM、Attention、Softmax、ApproxGELU、Residual 與 LayerNorm，可同時觀察運算量較大的 operator 與記憶體存取成本。[5]

2. 系統設計與實作

本專題大致保留 AITemplate 原有 frontend 與 graph compilation 流程，新增 x86 CPU target 與對應 backend code generator，使模型可產生 CPU C++ code 並編譯成 shared library。主要 FP32 Fully Connected / GEMM 與 Softmax 路徑整合 XNNPACK；Attention、LayerNorm 與其他 BERT-specific operations 則由 CPU backend 產生對應實作。XNNPACK 支援 x86/x86-64，並提供多種神經網路 low-level primitives，因此適合作為本 backend 的主要 kernel library。[4]

完成基本 CPU execution 後，profiling 顯示額外成本主要來自 static weight preparation、intermediate memory traffic，以及 operator boundary 造成的額外 tensor pass。後續最佳化因此以減少 inference 過程中的重複工作與記憶體存取為主。

表 1 主要效能問題與本專題實作方式

問題	實作方式
Fully Connected weights 在 inference 期間固定	Static weight prepacking：預先建立並快取 XNNPACK Fully Connected context，以固定 cache ID 辨識 static weights

GEMM、GELU、Residual、LayerNorm 間產生 temporary buffers	Scratch reuse / elimination、in-place ApproxGELU、Residual + LayerNorm overlay
Attention 的 Q scaling 需要額外讀寫 Q tensor	將 Q scaling 融合到 QKV permutation / copy
CPU operator 需要有效率的底層 kernel	主要 FP32 operators 整合 XNNPACK

2.1 Static weight prepacking

BERT 的 Fully Connected weights 在模型載入後保持固定，因此沒有必要在每次 inference 重複建立相同的 weight-dependent state。本專題在 generated operator 中加入 prepack path，並以 tensor name 產生 deterministic cache ID，使同一組 static weights 可重用已建立的 XNNPACK context，而不依賴 raw pointer 的位址。

此外，packed state 建立完成後，部分不再需要的 raw constant storage 可在後續釋放。這項處理對參數量較大的 BERT-large 與 Megatron-BERT 較有幫助，也與後續量測到的 peak RSS 差異相符。

2.2 Memory reuse 與 operator fusion

CPU inference 中，即使某個 operation 的計算量不高，只要需要額外讀寫一次大型 tensor，仍可能造成可觀察的時間與記憶體成本。因此針對 BERT 的實際 execution path 減少 temporary materialization。

ApproxGELU 在一般 BERT path 中直接於 GEMM output 上執行，不再另外配置 activation scratch。Residual + LayerNorm path 則讓 GEMM+bias 先寫入最終 buffer，再於同一 buffer 完成 residual addition 與 normalization，以移除原本額外的 GEMM-sized scratch space。

Attention 原本依序執行 QKV GEMM、QKV permutation、Q scaling，再進入 QK^T 。修改後在 QKV permutation 複製 Q 的同時完成 scaling，後續 QK^T 可直接使用處理後的資料，避免再對整個 Q tensor 進行一次獨立 pass。

3. 實驗設計與結果

實驗平台為 Intel Core i9-12900H，固定使用 P-cores；batch size 為 1、sequence length 為 128，資料型態為 FP32。比較對象為 PyTorch CPU inference，測試 BERT-base、BERT-large 與 Megatron-BERT 1.3B。Latency 以多次 isolated rounds 的 median-of-medians 統計，並使用 deterministic synthetic FP32 weights，使 AITemplate 與 PyTorch 使用相同 tensor。

表 2 AITemplate x86 backend 與 PyTorch CPU inference latency

Model	Threads	AITemplate	PyTorch	Speedup
BERT-base	1	207.29 ms	230.94 ms	1.114×
BERT-base	4	69.07 ms	75.24 ms	1.089×
BERT-large	1	725.25 ms	772.61 ms	1.065×
BERT-large	4	255.09 ms	270.90 ms	1.062×
Megatron-BERT 1.3B	1	2835.89 ms	2889.43 ms	1.019×
Megatron-BERT 1.3B	4	974.21 ms	992.63 ms	1.019×

BERT-base 的 1-thread latency 約降低 10.2%，4-thread 約降低 8.2%；BERT-large 分別約降低 4.4% 與 5.8%。Megatron-BERT 1.3B 在 1-thread 下約降低 1.9%，但改善幅度已明顯縮小。

表 3 Median Peak RSS

Model	Threads	AITemplate	PyTorch	Memory Reduction
BERT-base	1	607.34 MiB	684.87 MiB	12.7%
BERT-base	4	610.46 MiB	691.11 MiB	13.2%
BERT-large	1	1556.72 MiB	1934.70 MiB	19.5%
BERT-large	4	1556.94 MiB	1935.22 MiB	19.5%
Megatron-BERT 1.3B	1	5160.11 MiB	5540.12 MiB	6.9%
Megatron-BERT 1.3B	4	5160.09 MiB	5539.65 MiB	6.9%

Peak RSS 在 BERT-base 上約降低 13%，BERT-large 約降低 19.5%，Megatron-BERT 約降低 6.9%。此結果與 static weight storage 與 intermediate buffers 的最佳化方向一致；不過目前尚未完成逐項 ablation，因此無法將 memory reduction 完全歸因於單一修改。

4. 討論

從三種模型的結果可看出，模型規模增加後，移除 packing、scratch 與額外 tensor pass 所帶來的 latency 改善逐漸縮小。BERT-base 與 BERT-large 仍能觀察到穩定的改善；到了 Megatron-BERT 1.3B，profiling 顯示執行時間更集中在大型 GEMM，因此 graph-level memory optimization 在整體延遲中的占比下降。

這代表目前 CPU backend 的主要瓶頸已逐漸從周邊 overhead 轉向 matrix multiplication kernel 本身。oneDNN Graph Compiler 也採用 optimized primitives 與 compiler-level optimization 並行的方式，以同時處理高成本 kernel 與 graph-level overhead。[3] 對本專題而言，後續若要在更大型模型上取得明顯加速，除了維持既有的 memory 與 fusion 最佳化外，還需要進一步處理 GEMM kernel、資料型態或 shape-specific tuning。

整體結果顯示，AITemplate 原有的 graph 與 code-generation 架構可延伸至 x86 CPU，且編譯階段已知的 static weights、tensor shape 與 graph dependency 可實際用於減少 weight preparation、buffer allocation 與額外 tensor pass。

5. 研究限制與結論

目前 backend 仍屬實驗性實作，主要支援 FP32 與 BERT-family 所需 operators，尚未形成完整的 general-purpose CPU backend；部分 upstream frontend 與 runtime interface 也仍保留 GPU assumptions。此外，目前實驗集中在 Intel Core i9-12900H、batch size 1 與 sequence length 128，尚未涵蓋不同 CPU microarchitecture、batch size、sequence length 與其他 Transformer architectures。

本專題完成 AITemplate x86 CPU backend 的基本 code generation 與 BERT execution path，並實作 static weight prepacking、scratch buffer reuse / elimination、in-place ApproxGELU、Residual + LayerNorm overlay，以及 Q scaling 與 QKV permutation fusion。BERT-base 與 BERT-large 的 latency 均低於 PyTorch CPU baseline，BERT-large 亦觀察到較明顯的 peak RSS 降低；模型擴大至 Megatron-BERT 後，改善幅度縮小，profiling 顯示大型 GEMM 已成為主要瓶頸。後續工作可進一步針對 GEMM、低精度資料型態與更多 Transformer 模型進行實驗。

參考文獻

- [1] Meta AI, “Faster, more flexible inference on GPUs using AITemplate, a revolutionary new inference engine,” Oct. 3, 2022.
- [2] T. Chen et al., “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning,” in 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018, pp. 578–594.
- [3] J. Li et al., “oneDNN Graph Compiler: A Hybrid Approach for High-Performance Deep Learning Compilation,” in 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2024, pp. 460–470.

- [4] Google, “XNNPACK: High-efficiency floating-point neural network inference operators for mobile, server, and Web,” GitHub repository.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in NAACL-HLT, 2019, pp. 4171–4186.